

哈尔滨工业大学

编译系统实验报告（三）

姓名	曹云涵
学号	2021113258
学院	未来技术学院
教师	单丽莉

2024 年 5 月

一、实验目的

- 巩固对中间代码生成的基本功能和原理的认识。
- 能够基于语法制导翻译的知识进行中间代码生成。
- 掌握类高级语言中基本语句所对应的语义动作。

二、实验环境

- WSL2-Ubuntu_22.04
- GCC 11.4.0
- Flex 2.6.4
- Bison 3.8.2

三、实验过程

- 中间代码的作用：

经过了词法分析、语法分析和语义分析，我们得到了一棵语法分析树，接下来我们将以此为输入进行源代码的翻译，中间代码是源代码和目标代码之间的桥梁，它会将复杂的高级语言翻译成简单并且易懂的三地址码。然后后续将以这个三地址码为进一步分析的依据，生成目标代码。

- 翻译模式：

我们将程序中不同的语句进行分类，包括基本表达式，基本语句，声明语句，条件语句，调用语句，数据和结构体的赋值语句等语句。我们为每种语句，每个主要的语法单元设计相应的翻译函数。

- 实验中用到的数据结构：

本实验中我们需要在实验 2 的基础上额外实现两个数据结构：分别是操作数结构体以及中间代码结构体。前者的作用是保存每个操作数的类型和数值，用来完成中间代码的转换；后者的作用是保存生成的中间代码方便输出。

```
// 操作数结构体
struct Operand
{
    enum OperandKind kind; // 操作数的类型（变量、常量、地址等）
    int u_int;             // 存储整型数据，例如常量值或变量的索引
    char *u_char;          // 存储字符串数据，如变量名或函数名
    Type type;             // 操作数的数据类型信息，来自语义分析的结果
};
```

图 1：操作数结构体

```

struct InterCode_
{
    enum InterCodeKind kind;
    union
    {
        // LABEL, FUNCTION, GOTO, RETURN, ARG, PARAM, READ, WRITE
        struct
        {
            Operand op;
        } operand_1;
        // ASSIGN, CALL, ADDRESS1, ADDRESS2, ADDRESS3
        struct
        {
            Operand op1, op2;
        } operand_2;
        // ADD, SUB, MUL, DIV
        struct
        {
            Operand result, op1, op2;
        } operand_3;
    };
};

// IF
struct
{
    Operand x, relop, y, z;
} operand_if;
// DEC
struct
{
    Operand op;
    int size;
} operand_dec;
};
InterCode prev;
InterCode next;
};

```

图 2：中间代码结构体

其中中间代码这个结构体中，我采用了双向链表来进行前后节点之间的关联。

4. 翻译过程：

全部的语法单元以及基本表达式等内容均对应了一个翻译函数。我们在定义操作的时候先定义一个操作数，然后赋予字段值。然后我们调用中间代码生成函数来生成中间代码，并将其进行存储，利用上文中提到的结构体。然后在最后进行统一输出即可。

下图分别为中间代码生成函数以及一个翻译函数的例子。

```

// 中间代码生成函数_1
// 创建中间代码结构体的函数，接受中间代码的类型和一系列可变参数
InterCode create_intercode(enum InterCodeKind kind, int intercode_type, ...) {
    // 动态分配内存以存储一个InterCode结构体
    InterCode ic = (InterCode)malloc(sizeof(struct InterCode_));
    // 设置中间代码的种类
    ic->kind = kind;

    // 初始化va_list用于访问可变参数列表
    va_list vallist;
    va_start(vallist, (intercode_type % 10)); // 开始访问可变参数列表

    // 根据intercode_type的值，决定如何从参数列表中读取数据并填充到InterCode结构体中
    if (intercode_type == INTERCODE_1) {
        // 对于INTERCODE_1类型，期望有一个操作数
        ic->operand_1.op = va_arg(vallist, Operand); // 从参数列表中获取一个Operand
    }
    else if (intercode_type == INTERCODE_2) {
        // 对于INTERCODE_2类型，期望有两个操作数
        ic->operand_2.op1 = va_arg(vallist, Operand); // 从参数列表中获取第一个Operand
        ic->operand_2.op2 = va_arg(vallist, Operand); // 从参数列表中获取第二个Operand
    }
    else if (intercode_type == INTERCODE_3) {
        // 对于INTERCODE_3类型，期望有三个操作数（一个结果和两个输入）
        ic->operand_3.result = va_arg(vallist, Operand); // 从参数列表中获取结果操作数
        ic->operand_3.op1 = va_arg(vallist, Operand); // 从参数列表中获取第一个输入操作数
        ic->operand_3.op2 = va_arg(vallist, Operand); // 从参数列表中获取第二个输入操作数
    }
    else if (intercode_type == INTERCODE_IF) {
        // 对于INTERCODE_IF类型，期望有四个操作数（用于条件跳转）
        ic->operand_if.x = va_arg(vallist, Operand); // 条件左边的操作数
        ic->operand_if.relop = va_arg(vallist, Operand); // 关系操作符
        ic->operand_if.y = va_arg(vallist, Operand); // 条件右边的操作数
        ic->operand_if.z = va_arg(vallist, Operand); // 跳转的标签
    }
    else if (intercode_type == INTERCODE_DEC) {
        // 对于INTERCODE_DEC类型，期望有一个操作数和一个整数（用于数组或内存分配）
        ic->operand_dec.op = va_arg(vallist, Operand); // 操作数，通常是变量
        ic->operand_dec.size = va_arg(vallist, int); // 数组或分配的大小
    }
}

```

图 3：中间代码生成函数

```

// 比较语句的翻译
void translate_compst(Treenode now)
{
    // CompSt -> LC DefList StmtList RC
    // 翻译复合语句（compound statement），复合语句由大括号包围，包含一系列定义和语句。

    // 获取deflist节点，这通常是复合语句中的第一个定义列表。
    // 复合语句结构为：左大括号（LC），定义列表（DefList），语句列表（StmtList），右大括号（RC）。
    // "firstchild" 指向左大括号后的第一个元素，通常是一个定义列表或语句列表（如果没有定义）。
    Treenode deflist = now->firstchild->nextbro;

    // 获取stmtlist节点，它紧跟在deflist之后。
    // 在语法树中，定义列表后面直接是语句列表。
    Treenode stmtlist = deflist->nextbro;

    // 调用translate_DefList处理定义列表。
    // 这个函数负责遍历定义列表中的所有定义，并为每个定义生成相应的中间代码。
    // 例如，变量定义会生成用于分配存储空间的中间代码。
    translate_Deflist(deflist);

    // 调用translate_StmtList处理语句列表。
    // 这个函数遍历语句列表中的所有语句，并为每个语句生成相应的中间代码。
    // 这包括控制流语句（如if, while），表达式语句等。
    translate_Stmtlist(stmtlist);

    // 函数执行完后返回。在C语言中，复合语句不直接产生值，因此没有返回值。
    return;
}

```

图 4：比较语句的翻译过程

在这段代码中，我们首先对于语句进行成分划分（见函数第一行注释），然后我们在语法分析树中将这部分依次找到，然后将能直接进行翻译的部分进行翻译，不能直接翻译的部分进行递归翻译即可。

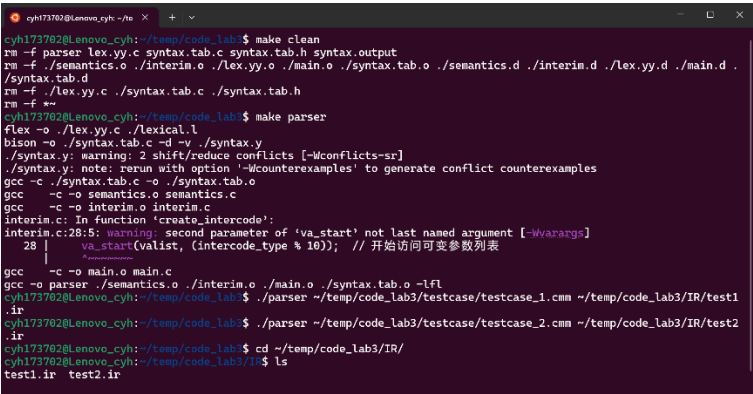
四、 实验的操作流程

在本实验中，仍然编写了 MakeFile 文件进行实验的运行。

首先采用 make parser 命令生成中间代码生成器。

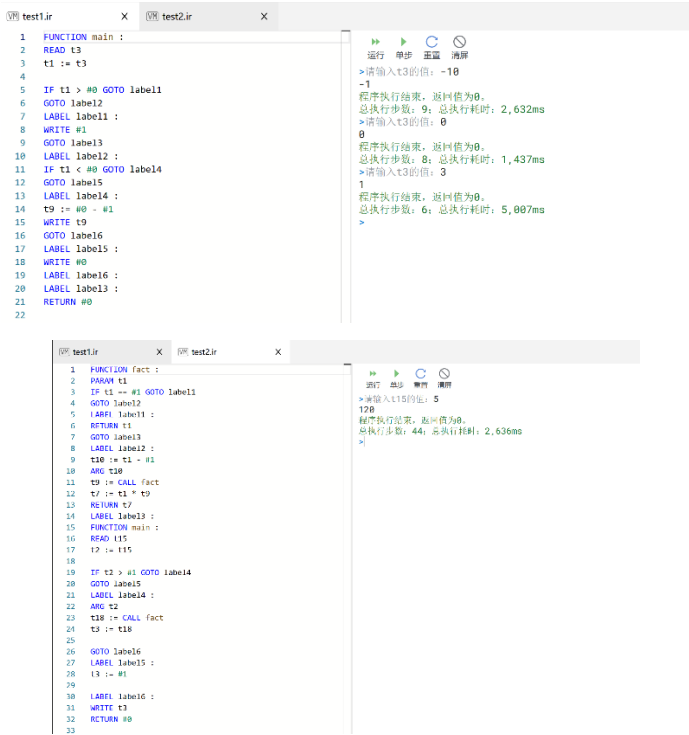
然后使用命令 ./parser testcase_1.cmm test1.ir 生成对于 testcase_1.cmm 代码的中间代码翻译，并存储在 test1.ir 文件中，最终使用指导书提供的线上 IR 虚拟机，并利用其检验中间代码生成的正确性。命令运行的结果，生成的中间代码在虚拟机中的运行情况均在下一部分中进行体现。

五、 实验的运行结果



```
cyhl73702@lenovo_cyh:~/temp/code_lab3$ make clean
rm -f parser lex.yy.c syntax.tab.c syntax.tab.h syntax.output
rm -f ./semantics.o ./interim.o ./lex.yy.o ./main.o ./syntax.tab.o ./semantics.d ./interim.d ./lex.yy.d ./main.d ./syntax.tab.d
rm -f ./lex.yy.c ./syntax.tab.c ./syntax.tab.h
rm -f **
cyhl73702@lenovo_cyh:~/temp/code_lab3$ make parser
flex -o ./lex.yy.c ./lexical.l
bison -o ./syntax.tab.c -d -y ./syntax.y
./syntax.y: warning: 2 shift/reduce conflicts [-Wconflicts-sr]
./syntax.y: note: run with option '-Wcounterexamples' to generate conflict counterexamples
gcc -c ./syntax.tab.c -o ./syntax.tab.o
gcc -c -o semantics.o semantics.c
gcc -c -o interim.o interim.c
interim.c: In function 'create_intercode':
interim.c:28:5: warning: second parameter of 'va_start' not last named argument [-Wvarargs]
   28 |     va_start(valist, (intercode_type % 10)); // 开始访问可变参数列表
       |     ~~~~~^
gcc -c -o main.o main.c
gcc -o parser ./semantics.o ./interim.o ./main.o ./syntax.tab.o -lfl
cyhl73702@lenovo_cyh:~/temp/code_lab3$ ./parser ~/temp/code_lab3/testcase/testcase_1.cmm ~/temp/code_lab3/IR/test1.ir
cyhl73702@lenovo_cyh:~/temp/code_lab3$ ./parser ~/temp/code_lab3/testcase/testcase_2.cmm ~/temp/code_lab3/IR/test2.ir
cyhl73702@lenovo_cyh:~/temp/code_lab3$ cd ~/temp/code_lab3/IR/
cyhl73702@lenovo_cyh:~/temp/code_lab3/IR$ ls
test1.ir test2.ir
```

图 5：命令执行结果



The image shows two screenshots of the IR virtual machine execution results. The top screenshot shows the execution of test1.ir, which includes a main function with various control flow statements like IF, GOTO, and LABEL. The bottom screenshot shows the execution of test2.ir, which includes a fact function with arithmetic operations and a main function that calls fact.

test1.ir Execution Results:

- 程序执行结束，返回值为0。
- 总执行步数：9；总执行耗时：2,632ms
- 请输入t3的值：-10
- 1
- 程序执行结束，返回值为0。
- 总执行步数：8；总执行耗时：1,437ms
- 请输入t3的值：0
- 0
- 程序执行结束，返回值为0。
- 总执行步数：8；总执行耗时：1,437ms
- 请输入t3的值：3
- 3
- 程序执行结束，返回值为0。
- 总执行步数：6；总执行耗时：5,007ms

test2.ir Execution Results:

- 程序执行结束，返回值为0。
- 总执行步数：44；总执行耗时：2,636ms
- 请输入t3的值：5
- 5
- 程序执行结束，返回值为0。
- 总执行步数：44；总执行耗时：2,636ms

图 6：样例 1、2 运行与验证结果

六、 实验总结&心得

通过本次实验，我对于中间代码的生成过程，实现方法有了进一步的认识，并且在小程序执行中间代码的运行过程中深化了中间代码在编译系统中桥梁的作用。